

Agregados en PostgreSQL

En esta práctica vamos a ver como el modelo de datos de PostgreSQL, que proporciona soporte tanto para tipos de datos compuestos como para arrays, permite el almacenamiento y gestión de datos agregados, es decir, datos que no están en primera forma normal.

Instalación del entorno de trabajo (Docker - PostgreSQL 16 - DBeaver)

Docker y creación del contenedor

El software se descarga de la página de Docker (<https://www.docker.com/>), donde también tienes instrucciones de instalación para tu sistema operativo (<https://docs.docker.com/desktop/setup/install/windows-install/>). En Windows, la recomendación es instalar Docker sobre **WSL**, ejecutando el comando `wsl --install`. Podremos utilizar WSL para instalar paquetes de Ubuntu directamente en nuestro Windows.

Creemos un contenedor Docker en el que tendremos una instalación del sistema gestor de bases de datos **PostgreSQL**. Lo primero que debemos hacer es descargar el **Dockerfile** de PostgreSQL del aula virtual y crear la imagen que nos permitirá generar contenedores usando el siguiente comando.

Si abres el Dockerfile con un editor de texto, verás el siguiente contenido, que incluye los comandos a ejecutar para realizar una instalación de **PostgreSQL 16**. También podrás ver instrucciones que configuran el servidor, por ejemplo, añadiendo un usuario al sistema operativo y también a la base de datos, y creando una base de datos por defecto para él.

Para editar archivos de texto en Windows se recomienda la instalación de un editor avanzado, como por ejemplo **Notepad++** (<https://notepad-plus-plus.org/>). En Linux tienes editores similares disponibles en tu distribución.

?

```
FROM ubuntu:latest
EXPOSE 5432
RUN useradd -rm -d /home/alumnobd -s /bin/bash -g root -G sudo -p pwalumnobd alumnobd
RUN apt update
RUN apt install -y postgresql-16
RUN echo "host all all all md5" >> /etc/postgresql/16/main/pg_hba.conf
RUN pg_conftool 16 main set listen_addresses '*'
RUN service postgresql start && \
    su postgres -c "psql -c \"ALTER USER postgres WITH PASSWORD 'pwalumnobd'\" \"&&
    su postgres -c \"psql -c \"CREATE USER alumnobd WITH superuser createdb createrole login password
'pwalumnobd'\" \"&&
    su postgres -c \"psql -c \"CREATE Database alumnobd WITH owner = alumnobd\" \"
CMD service postgresql start;su alumnobd
```

Para poder crear contenedores, lo primero que tenemos que hacer es **crear una imagen a partir del Dockerfile**. En Windows usamos **PowerShell** o el **Command Prompt (cmd)**. En Linux o Mac, una **terminal**.

```
docker buildx build --platform=linux/amd64 -t postgresql:xine-grei "ruta a la carpeta donde está el dockerfile"
```

El parámetro **-t** indica el nombre que tendrá la imagen. La ruta indicada debe referenciar la **carpeta** en la que está el Dockerfile (no directamente el Dockerfile).

Una vez creada la imagen, deberíamos poder verla en la lista de imágenes a través de la interfaz gráfica de Docker.

Una vez creada la imagen, ya podemos crear contenedores con ella. En este caso, vamos a crear **un solo contenedor** utilizando el siguiente comando:

```
docker run --name=xine-grei-pgsq -dti -p 35432:5432 --platform=linux/amd64 -v c:\temp:/home/alumnobd/host-temp
postgresql:xine-grei
```

- El parámetro **--name** permite especificar el nombre del contenedor.
- El parámetro **-p** asocia un puerto local a un puerto expuesto desde dentro del contenedor. En este caso, enlazamos el **puerto 35432 de la máquina host** con el **puerto 5432 del contenedor**, donde está escuchando PostgreSQL.

- Finalmente, el parámetro **-v** asocia una carpeta de nuestro host con una carpeta del sistema de archivos del contenedor, lo que permite **compartir archivos entre las dos máquinas**.

Con los siguientes comandos puedes parar, iniciar y pausar la ejecución del contenedor.

También lo puedes eliminar, pero ten en cuenta que, si lo eliminas, **se perderán todos los datos** que tengas en su espacio de disco, incluido el contenido de la base de datos. En Windows, también puedes realizar todas estas tareas desde la propia interfaz gráfica de Docker.

```
docker start xine-grei-pgsql (para iniciar o contenedor despois de telo parado)
docker stop xine-grei-pgsql (para parar o contenedor)
docker restart xine-grei-pgsql (para reiniciar o contenedor)
docker pause xine-grei-pgsql (para pausar a execución de todos os procesos do contenedor)
docker unpause xine-grei-pgsql (para reanudar a execución de todos os procesos do contenedor)
docker rm xine-grei-pgsql (para eliminar o contenedor)
```

Para abrir una **terminal dentro del contenedor**, que te permita ejecutar comandos, puedes usar:

```
docker exec -ti xine-grei-pgsql bash
```

Necesitarás hacer esto, por ejemplo, para **cargar los datos en la base de datos** usando llamadas a **psql**. Una vez dentro, cambia al usuario **alumnobd**, accede a su carpeta home y luego a la carpeta compartida donde estén los archivos que necesites:

```
su alumnobd
cd
cd host-temp
```

El cliente **DBeaver** se puede descargar e instalar desde la siguiente URL: <https://dbeaver.io/>

Una vez instalado, puedes configurar una conexión contra la base de datos **"alumnobd"**, que ya tienes creada en tu servidor PostgreSQL dentro del contenedor.

Para ello:

1. Elige la opción **Database / New Database Connection**, o pulsa el botón con el icono de enchufe en la parte izquierda del panel de botones.

2. Selecciona el gestor **PostgreSQL** (el que tiene el símbolo de un elefante azul) y pulsa **Siguiente**.

3. En el formulario introduce:

- **Host:** `localhost`
- **Port:** `35432`
- **Database:** `alumnobd`
- **Usuario:** `alumnobd`
- **Contraseña:** `pwalumnobd`

4. En la pestaña **PostgreSQL**, marca la opción **"Show all databases"**.

5. Pulsa **Finish**.

La primera vez que uses DBeaver para conectar con una base de datos PostgreSQL es posible que tengas que instalar el **driver JDBC para PostgreSQL**.

DBeaver es un cliente programado en Java, y JDBC es el estándar de conectores desde Java hacia cualquier base de datos.

Ahora ya puedes **crear nuevas bases de datos usando DBeaver** dentro del servidor que tienes en el contenedor.

Puedes crear una base de datos "xine" para continuar con este tutorial.

Tipos de dato en PostgreSQL

Numéricos

- Enteros: `smallint`, `integer`, `bigint`
- Precisión arbitraria: `numeric(precision)`, `numeric(precision, scale)`
- Punto flotante: `real`, `double precision`

Caracteres: `varchar(n)`, `char(n)`, `text`

Otros: Monedas, binarios, tiempo, boolean, etc.

Tipos de dato ARRAY

Además, de los tipos básicos que dan soporte para las estructuras de datos del modelo relacional, PostgreSQL soporta también tipos de dato ARRAY. Los arrays pueden tener cualquier número de dimensiones y se soportan dos formas de sintaxis. La forma con la palabra clave ARRAY es la usada por el estándar SQL, pero solo se puede usar con arrays unidimensionales.

```
drop table if exists empleado;
create table empleado (
  id int primary key,
  nombre varchar(100),
  calle varchar(255),
  numero varchar(5),
  cp varchar(5),
  localidad varchar(100),
  paga_mensual numeric(8,2)[12],
  paga_mensual_trienio numeric(8,2)[3][12]
);

drop table if exists empleado;
create table empleado (
  id int primary key,
  nombre varchar(100),
  calle varchar(255),
  numero varchar(5),
  cp varchar(5),
  localidad varchar(100),
  paga_mensual numeric(8,2) ARRAY[12],
  paga_mensual_trienio numeric(8,2) [3][12]
);
```

El índice de la dimensión se proporciona solo a efectos de documentación. El con lo que el atributo `paga_mensual` del ejemplo de arriba podría declararse igualmente como `paga_mensual numeric(8,2) []`. Los arrays pueden tener varias dimensiones, pero tampoco se verifican las dimensiones en la declaración.

Los literales de tipo ARRAY se pueden expresar usando un string que contiene el array delimitado por símbolos de llaves. Pero también se puede usar la palabra clave ARRAY y los corchetes como se puede ver en siguiente sentencia de inserción.

```
insert into empleado values
(1, 'Miguel Angel Pintor', 'Via del Corso', '14', '23452', 'Roma',
 '{23000, 23500, 23800, 23800, 23500, 22900, 23200, 46000, 23100, 23700, 23500, 46100}',
 array[
  array[23000, 23500, 23800, 23800, 23500, 22900, 23200, 46000, 23100, 23700, 23500, 46100],
  array[23900, 12534, 22354, 43233, 12324, 12324, 12134, 21313, 23245, 13346, 23523, 23246],
  array[21233, 12135, 23234, 23342, 21342, 23412, 12341, 12352, 21345, 12355, 34521, 52445]
]);
```

En las siguientes inserciones se puede ver como no se verifica ni el número de dimensiones ni el número de elementos por dimensión. Lo que sí ha de cumplirse es la coherencia dentro de cada array. Así, en las siguientes inserciones, sólo la última produce un fallo en el primero de sus arrays.

```
insert into empleado values
(2, 'Leonardo Ingenioso', 'Via Appia Antica', '5', '24987', 'Roma',
 array[12,24,18,12,143,56,34,34,64,456,34,756,34,14,23],
 '{2,32,43,53},{23,34,354,35},{23,243,34,34}}');

insert into empleado values
(3, 'Donato di Niccolo di Betto Bardi', 'Via dei Coronari', '28', '23452', 'Roma',
 '{{23000, 23500, 23800, 23800}, {23500, 22900, 23200, 46000}, {23100, 23700, 23500, 46100}}',
 '{234,232}');

insert into empleado values
(4, 'Rafael Sanzio', 'Via Margutta', '34', '23456', 'Roma',
 array[array[12,12,4234,23,123,24,23],array[124,434,45,345]],
 '{2436,346723,234623}');
```

La última inserción produce un error, dado que los dos subarrays no tienen el mismo tamaño, y esto produciría un array de dimensión 2 con menos columnas en una fila que en la siguiente.

Los arrays se pueden consultar usando índices en las expresiones.

```
select id, nombre, paga_mensual[7] from empleado

select id, nombre, paga_mensual[2][3] from empleado

select id, nombre, paga_mensual[4:8] from empleado

select id, nombre, paga_mensual
from empleado
where paga_mensual[8]>13423
```

Los arrays se pueden convertir en tablas usando el operador UNNEST en la cláusula FROM.

```
select id, nombre, paga
from empleado, unnest(paga_mensual) as pagas(paga)

select id, nombre, sum(paga) as paga_anual
from empleado, unnest(paga_mensual) as pagas(paga)
where paga > 234
group by id, nombre
having sum(paga) > 123223
```

Se puede generar una columna con el índice del array vinculado a cada elemento.

```
select id, nombre, paga, mes
from empleado,
     unnest(paga_mensual) with ordinality as pagas(paga,mes)
where id = 1
order by mes desc

select id, nombre, paga,
       (mestotal-1)/12+1 as ano,
       (mestotal-1)%12+1 as mes
from empleado, unnest(paga_mensual_trienio)
               with ordinality as pagas(paga,mestotal)
where id = 1
order by ano, mes
```

Se puede generar un array a partir de una subconsulta

```
select id, nombre,
       ARRAY(select nombre
             from empleado e2
             where e1.localidad = e2.localidad
             order by id) as mismalocalidad
from empleado e1
where id = 1
```

Se puede agregar un conjunto de valores para generar un array usando la función de agregado `array_agg`. Se puede especificar también un orden en la función de agregado. Esta función también permite concatenar arrays ya existentes en un array de dimensión superior, siempre que los arrays tengan el mismo número de elementos.

```
select localidad, array_agg(nombre) as empleados
from empleado
group by localidad

select localidad, array_agg(nombre order by id) as pagas
from empleado
group by localidad
```


Tipos de dato Compuestos

En PostgreSQL, un campo del resultado de una consulta puede tener un tipo de dato ROW, es decir, un tipo de dato que contiene varias columnas anidadas. Un campo de tipo ARRAY, puede tener también como tipo de dato de cada uno de sus elementos un tipo ROW. Sin embargo, al contrario de lo que ocurren en el estándar de SQL, no se pueden crear directamente estructuras de tabla que tengan columnas de estos tipos ROW si no se crea previamente el tipo con una instrucción CREATE TYPE.

```
select id, nombre,  
       (calle, numero, cp, localidad) as direccion,  
       paga_mensual  
from empleado  
  
select localidad,  
       array_agg((calle, numero, cp, localidad)) as direcciones  
from empleado  
group by localidad  
  
create table empleado2 as  
select id, nombre,  
       (calle, numero, cp, localidad) as direccion,  
       paga_mensual  
from empleado
```

La creación de la tabla falla porque no existe un tipo compuesto para la dirección. La creación de un tipo compuesto se realiza con la sentencia CREATE TYPE de forma similar a como se crea la estructura de una tabla, con la diferencia de que no se especifican restricciones. La creación de la tabla falla porque no existe un tipo compuesto para la dirección. La creación de un tipo compuesto se realiza con la sentencia CREATE TYPE de forma similar a como se crea la estructura de una tabla, con la diferencia de que no se especifican restricciones.

```
drop table if exists empleado2;
drop type if exists tipo_direccion;
create type tipo_direccion as (
  calle varchar(255),
  numero varchar(5),
  cp varchar(5),
  localidad varchar(100)
);
create table empleado2 (
  id int primary key,
  nombre varchar(100),
  direccion tipo_direccion,
  paga_mensual numeric(8,2)[12]
);
insert into empleado2
select id, nombre,
       cast((calle, numero, cp, localidad) as tipo_direccion),
       paga_mensual
from empleado
```

Ejemplo de gestión de agregados en PostgreSQL

1. Inserta en la base de datos los datos proporcionados en el script “**tormentas.sql**”

```
psql -d xine -f tormentas.sql
```

2. En este punto deberías de tener tres tablas en la base de datos, **tormentas**, **regiones** y **regiones_afectadas**

3. Crea un tipo de datos compuesto (**pais**) que permita almacenar el conjunto de regiones de cada país. El tipo debe tener los siguientes campos:

- nombre: para almacenar el nombre del país.
- regiones: un array de elementos de tipo **varchar(100)** que almacena el conjunto de regiones del país.

```
create type pais as (  
    nombre varchar(100),  
    regiones varchar(100) ARRAY  
);
```

4. Crea una tabla (**tormentas1**) en la que se puedan almacenar las tormentas, con todos sus datos, pero también los países y regiones afectadas por cada tormenta.

```
drop table if exists tormentas1;  
create table tormentas1 (  
    temporada int,  
    nombre varchar(100),  
    inicio timestamp,  
    fin timestamp,  
    viento_max float,  
    viento_medio float,  
    paises pais ARRAY  
);
```

5. Inserta en esta nueva tabla los datos obtenidos de las tablas de tormentas y **regiones_afectadas**, usando una sentencia SQL de selección-inserción.

```
insert into tormentas1  
select *,  
    ARRAY(select cast((ra.pais, array_agg(ra.region)) as pais)  
          from regiones_afectadas ra  
          where t.temporada=ra.temporada  
            and t.nombre=ra.nombre  
          group by ra.pais  
          ) as paises  
from tormentas t
```

6. Realiza las siguientes consultas usando la nueva tabla **tormentas1**, y compara el código con el código que necesitarías para consultar lo mismo con las tablas **tormentas** y **regiones_afectadas**. Compara también los tiempos de ejecución de las consultas.

a. Obtén la lista de países afectados por el huracán KATRINA de la temporada 2005

```
select pais
from tormentas1 t, unnest(t.países) as pais (pais,regiones)
where nombre = 'KATRINA'
    and temporada = 2005

select distinct ra.pais
from tormentas t, regiones_afectadas ra
where t.nombre = ra.nombre
    and t.temporada = ra.temporada
    and t.temporada = 2005
    and t.nombre = 'KATRINA'
```

b. Obtén la lista de pares (país, región) afectados por el huracán KATRINA de la temporada 2005

```
select pais, region
from tormentas1 t,
    unnest(t.países) as pais (pais,regiones),
    unnest(pais.regiones) as region (region)
where nombre = 'KATRINA'
    and temporada = 2005

select ra.pais, ra.region
from tormentas t, regiones_afectadas ra
where t.nombre = ra.nombre
    and t.temporada = ra.temporada
    and t.temporada = 2005
    and t.nombre = 'KATRINA'
```

c. Obtén la lista de tormentas (con todos sus datos) que han afectado a la región de Florida de United States.

```
select t.*
from tormentas1 t,
     unnest(t.países) as pais (pais,regiones),
     unnest(pais.regiones) as region (region)
where pais = 'United States'
     and region = 'Florida'

select *
from tormentas1
where ('United States','Florida')
     in (select pais,region
         from unnest(países) as pais (pais,regiones),
         unnest(regiones) as region(region))

select t.*
from tormentas t, regiones_afectadas ra
where t.nombre = ra.nombre
     and t.temporada = ra.temporada
     and ra.pais = 'United States'
     and ra.region = 'Florida'
```

Gestión de datos JSON en PostgreSQL

PostgreSQL tiene tipos de datos específicos para gestionar documentos XML, documentos JSON y también para realizar búsquedas por palabras clave sobre texto (full text search). Aquí vamos a ver como podemos gestionar agregados representados con objetos JSON. En el manual de PostgreSQL hay descripciones del tipo de datos JSON y también de los operadores y funciones soportados para su gestión.

1. Importa datos de películas usando los siguientes comandos. La tabla “**películas**” resultante tendrá varios atributos de tipo json

```
CREATE TABLE peliculas (  
    id int4,  
    titulo text,  
    para_adultos bool,  
    coleccion json,  
    presupuesto int4,  
    generos json,  
    web text,  
    idioma_original text,  
    titulo_original text,  
    sinopsis text,  
    popularidad float8,  
    empresas_produccion json,  
    paises_produccion json,  
    fecha_emision date,  
    ingresos int8,  
    duracion float8,  
    idiomas_hablados json,  
    lema text NULL  
);  
  
psql -d xine -f peliculas.sql
```

2. Importa datos de créditos de las películas usando los siguientes comandos. La tabla “**creditos**” tendrá el reparto y el personal que participó en cada película, ambos descritos como estructuras json.

```
CREATE TABLE creditos (  
    pelicula int4,  
    reparto json,  
    personal json  
);  
  
psql -d xine -f creditos.sql
```

3. Obtén los nombres de las empresas productoras de la película con el presupuesto más alto

```
select empresa-->'name' as productora
from peliculas, json_array_elements(empresas_produccion) as e(empresa)
where presupuesto = (select max(presupuesto) from peliculas)
```

4. Obtén la lista de pares, (actor/actriz, personaje) de la película con el presupuesto más alto. Ordena el resultado por el orden en el que aparecen en el reparto (atributo “**order**”)

```
select ereparto-->'name' as actriz,
       ereparto-->'character' as personaje
from peliculas p, creditos c , json_array_elements(c.reparto) as r(ereparto)
where p.id = c.pelicula
      and p.presupuesto = (select max(presupuesto) from peliculas)
order by cast(ereparto-->'order' as integer)
```

5. Obtén el listado de películas dirigidas por “**Quentin Tarantino**” ordenado por la fecha de emisión. De cada película muestra el título, fecha de emisión y presupuesto.

```
select p.titulo, p.fecha_emision, p.presupuesto
from peliculas p, creditos c , json_array_elements(c.personal) as personal(empleado)
where p.id = c.pelicula
      and empleado-->'job' = 'Director'
      and empleado-->'name' = 'Quentin Tarantino'
order by p.fecha_emision
```

6. Para las diez películas con mayor presupuesto, obtén su título y un array de objetos JSON, cada uno con el nombre de un actor/actriz de reparto y el personaje que interpreta. Ordena este array por el orden en el que aparecen en el reparto (atributo “**order**”).

```
select p.titulo,  
       json_agg(json_build_object('nombre', ereparto->'name', 'personaje', ereparto->'character')  
                order by cast(ereparto->>'order' as integer)  
                ) as reparto  
from películas p, credits c,  
     json_array_elements(c.reparto) as reparto(ereparto)  
where p.id = c.pelicula  
group by p.titulo, p.presupuesto  
order by p.presupuesto desc  
limit 10
```

Última modificación: luns, 9 de febreiro de 2026, 19:26